B11508015 盧雅筠

Theoretical questions

1. Give a single intensity transformation function for spreading the intensities of an image so the lowest intensity is 0 and the highest is L-1.

⇒ linear contrast stretching

Piecewise-linear transformation : $g(r) = \begin{cases} 0, & r \le r_1 \\ \frac{r-r_1}{r_2-r_1}(s_2-s_1)+s_1, & r_1 < r < r_2 \\ L-1, & r \ge r_2 \end{cases}$

r：輸入像素灰階值，g(r)：輸出像素灰階值，r_1, r_2 輸入中定義的低、高臨界灰階值，s_1, s_2 希望輸出的低、高灰階值

∵ "single" intensity transformation function

∴ set $r_1 = f_{min}$, $r_2 = f_{max}$, $s_1 = 0$, $s_2 = L-1$

⇒ $g(r) = \frac{r - f_{min}}{f_{max} - f_{min}}(L-1)$ satisfy lowest intensity is 0 代 $r=f_{min}$, the highest is L-1 代 $r=f_{max}$ #

2. Show that subtracting the Laplacian from an image is proportional to unsharp masking. Use the definition for the Laplacian given in Eq.

$$\nabla^2 f = [f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1)] - 4f(x,y)$$

4-neighbor discrete Laplacian : $\nabla^2 f(x,y) = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$

subtract the Laplacian from the image : $g(x,y) = f(x,y) - \nabla^2 f(x,y)$

$\Leftrightarrow g(x,y) = f(x,y) - [f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)]$

$= 5f(x,y) - [f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1)]$

Unsharp masking : $f_{hp}(x,y) = f(x,y) - f_{lp}(x,y)$

$\Rightarrow g(x,y) \propto f_{hp}(x,y)$ 得證 #

Programming exercises (Send the source code)

1. 程式碼為 Figure 1，輸出的結果為 Figure 2。

```python
import numpy as np
import cv2
import matplotlib.pyplot as plt

def Histogram(img):
    hist = np.zeros(256) # vector

    for rows in img: # a row, nparray
        for element in rows:
            hist[element] += 1

    # Probability Density Function (PDF)
    PDF = hist/hist.sum() # Normalized into [0,1]

    return hist, PDF # vector

'''Cumulative Distribution Function(CDF)'''
def CDF(img):
    cdf = np.zeros(256) # vector

    _, pdf = Histogram(img)

    for idx, element in enumerate(pdf):
        if idx == 0:
            cdf[idx] += element
        else:
            cdf[idx] += (element + cdf[idx-1]) # 累加

    return cdf # vector

'''Histogram Equalization'''
def HistogramEqual(img_input,L):
    img_out = np.zeros(shape=img_input.shape)
    cdf = CDF(img_input) # vector

    for rows, element_in_col in enumerate(img_input):
        for cols, element in enumerate(element_in_col):
            img_out[rows, cols] = L*cdf[element] # The transform function
    return img_out

paths = ['./data/Fig1-1.bmp', './data/Fig1-2.bmp', './data/Fig1-3.bmp', './data/Fig1-4.bmp']
imgs = []
Hists = []
PDFs = []
His_Eqs = []
Eqs_hists = []

for p in paths:
    img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
    imgs.append(img)
    hist, pdf = Histogram(img)
    Hists.append(hist)
    PDFs.append(pdf)
    his_eq = HistogramEqual(img, 255)
    His_Eqs.append(his_eq)
    eq_hist, _ = Histogram(his_eq.astype(np.uint8))
    Eqs_hists.append(eq_hist)



fig, axes = plt.subplots(4, 5, figsize=(18, 16))

for i in range(4):
    axes[i, 0].imshow(imgs[i], cmap='gray', vmin=0, vmax=255)
    axes[i, 0].set_title(f'Fig1-{i+1}')
    axes[i, 0].axis('off')

    axes[i, 1].bar(range(len(Hists[i])), Hists[i], color='green', width=1.0)
    axes[i, 1].set_title('Histogram')
    axes[i, 1].set_xlim(0, 255)

    axes[i, 2].bar(range(len(PDFs[i])), PDFs[i], color='green', width=1.0)
    axes[i, 2].set_title('PDF')
    axes[i, 2].set_xlim(0, 255)

    axes[i, 3].imshow(His_Eqs[i], cmap='gray', vmin=0, vmax=255)
    axes[i, 3].set_title('Histogram Equalization Result')
    axes[i, 3].axis('off')

    axes[i, 4].bar(range(len(Eqs_hists[i])), Eqs_hists[i], color='green', width=1.0)
    axes[i, 4].set_title('Equalized Histogram')
    axes[i, 4].set_xlim(0, 255)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig('./hw2_1.png', dpi=300)
```

Figure 1

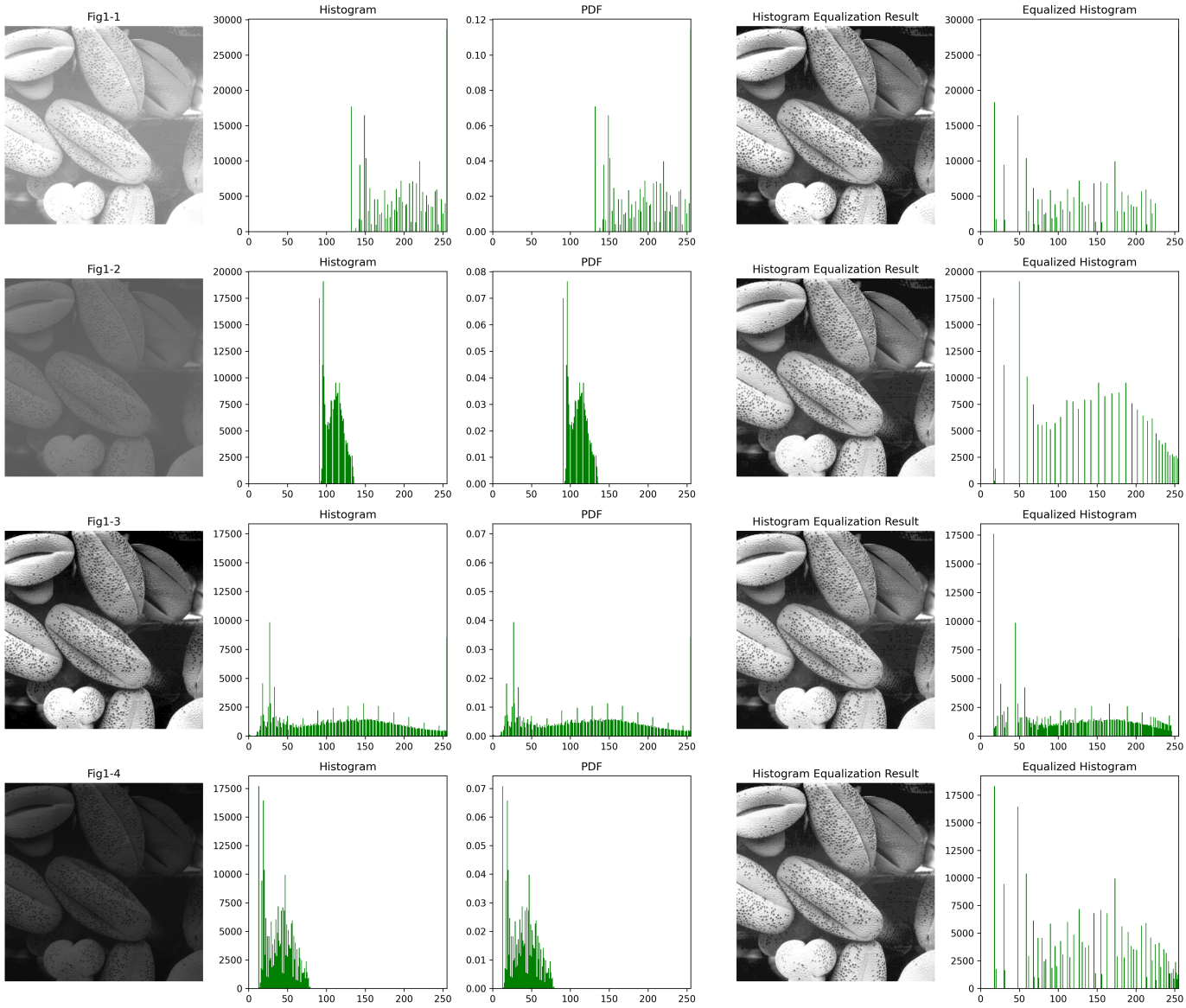
Fig1-1
Histogram
PDF
Histogram Equalization Result
Equalized Histogram
Fig1-2
Histogram
PDF
Histogram Equalization Result
Equalized Histogram
Fig1-3
Histogram
PDF
Histogram Equalization Result
Equalized Histogram
Fig1-4
Histogram
PDF
Histogram Equalization Result
Equalized Histogram

Figure 2

2. 程式碼為 Figure 3，輸出的結果為 Figure 4。

```python
import cv2
import matplotlib.pyplot as plt

m_sizes = [3, 5, 9, 15, 35]

img = cv2.imread('./data/Fig2-1.bmp', cv2.IMREAD_GRAYSCALE)

results = [img]
titles  = ['Original']
for m in m_sizes:
    out = cv2.blur(img, (m, m), borderType=cv2.BORDER_REFLECT)
    results.append(out)
    titles.append(f'm = {m}')

plt.figure(figsize=(6, 8))
for i, (res, title) in enumerate(zip(results, titles)):
    plt.subplot(3, 2, i + 1)
    plt.imshow(res, cmap='gray', vmin=0, vmax=255)
    plt.title(title)
    plt.axis('off')

plt.tight_layout()
plt.savefig('./hw2_2.png', dpi=300)
```

Figure 3

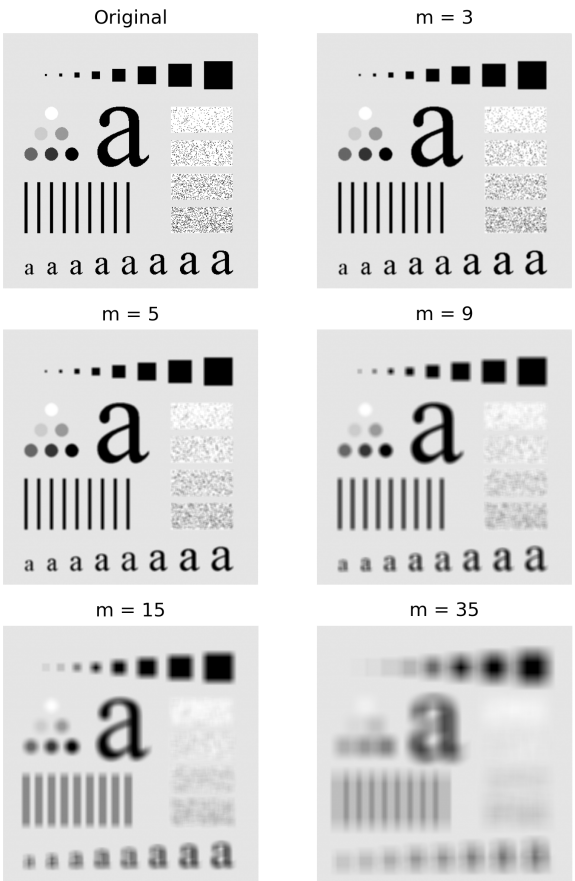

Figure 4

3. 程式碼為 Figure 5，輸出的結果為 Figure 6。

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('./data/Fig3-1.bmp', cv2.IMREAD_GRAYSCALE)

# Roberts Cross kernels
kx = np.array([[1, 0],
               [0, -1]], dtype=np.float32)
ky = np.array([[0, 1],
               [-1, 0]], dtype=np.float32)

# calculate gradients
gx = cv2.filter2D(img, cv2.CV_32F, kx)
gy = cv2.filter2D(img, cv2.CV_32F, ky)

# calculate gradient magnitude
grad = cv2.magnitude(gx, gy)
grad = np.clip(grad, 0, 255).astype(np.uint8)


plt.figure(figsize=(8, 6))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(grad, cmap='gray')
plt.title('Roberts Cross Gradient')
plt.axis('off')

plt.tight_layout()
plt.savefig('./hw2_3.png', dpi=300)
```

Figure 5

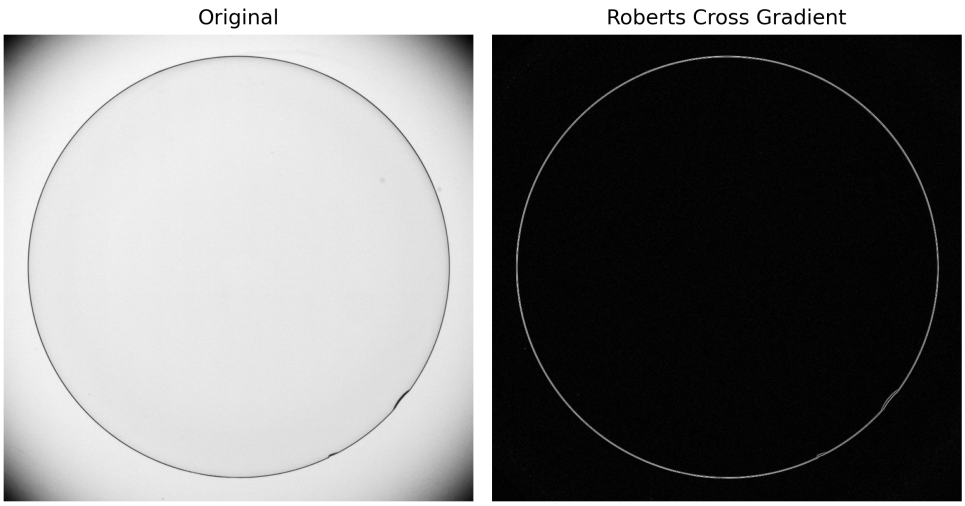

Figure 6

4. 程式碼為 Figure 7，輸出的結果為 Figure 8。

```python
import cv2, numpy as np, matplotlib.pyplot as plt

# (a): Raw image
a = cv2.imread('./data/Fig4-1.bmp', cv2.IMREAD_GRAYSCALE)

# (b): Laplacian of (a)
b = cv2.Laplacian(a, cv2.CV_32F, ksize=3)
b_norm = cv2.normalize(b, None, 0, 255, cv2.NORM_MINMAX)
b_norm = b_norm.astype(np.uint8)


# (c): Sharpened image obtained by adding (a) and (b)
c = cv2.convertScaleAbs(a.astype(np.float32) - b)

# (d): Sobel gradient of (a)
gx = cv2.Sobel(a, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(a, cv2.CV_32F, 0, 1, ksize=3)
d = cv2.convertScaleAbs(cv2.magnitude(gx, gy))

# (e): Smoothed with a 5*5 averaging filter of (d)
e = cv2.blur(d, (5, 5), borderType=cv2.BORDER_REFLECT)

# (f): Mask image formed by the product of (c) and (e)
f = cv2.convertScaleAbs((c.astype(np.float32) * e.astype(np.float32)) / 255.0)

# (g): Sharpened image obtained by the sum of (a) and (f)
g = cv2.add(a, f)

# (h): Applying a power-law transformation to (g), gamma = 0.5
h = (np.power(g.astype(np.float32)/255.0, 0.5) * 255.0 + 0.5).astype(np.uint8)


titles = ['(a) Original','(b) Laplacian','(c) a - b','(d) Sobel |∇f|',
          '(e) Smooth(d)','(f) c * e','(g) a + f','(h) Power-law γ=0.5']
imgs   = [a, b_norm, c, d, e, f, g, h]

plt.figure(figsize=(4, 12))
for i,(im,ti) in enumerate(zip(imgs, titles), 1):
    plt.subplot(4,2,i); plt.imshow(im, cmap='gray', vmin=0, vmax=255)
    plt.title(ti); plt.axis('off')
plt.tight_layout()
plt.savefig('./hw2_4.png', dpi=300)
```

Figure 7

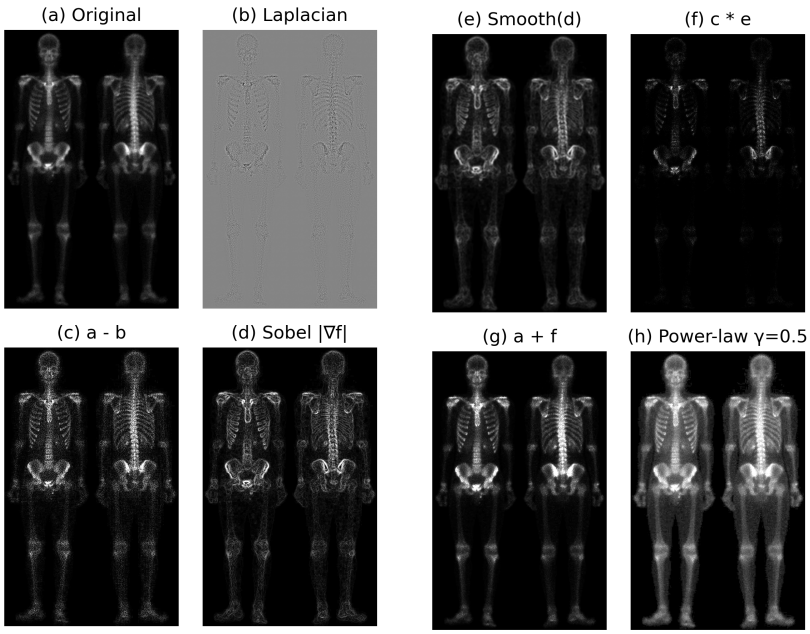

Figure 8